FORWARD DEPLOYED ENGINEER | CASE STUDY

Improving the MoveInSync Employee App Rating

What 500 low-rating reviews tell us, and how I would fix it in 30 days

Prepared by Tharun Teja
10 October 2026

Summary

The MoveInSync employee app has a 4.3 rating on Google Play, from about 1.15 lakh reviews (checked on 10 October 2026). A rating like that shows how people feel, but it does not show what to fix. So I wrote a Python script that collects public Google Play reviews, and I used it to pull a random sample of 500 one-star and two-star reviews. I tagged each review by topic and looked at what people complain about, when they posted, and how much they write.

What I found is simple. About half of the complaints are about the trip itself: booking, finding the vehicle, arrival time, alerts, and driver or vehicle problems. Very few are about how the app looks. Another third of the reviews could not be put in any topic, mostly because they are very short.

So my advice is to make the commute reliable first and let the rating follow. Section 5 is the main part of this report. It lays out who does what, what happens each week, how to release changes safely, and how to know whether they worked.

- Booking and scheduling is the biggest topic we could identify, at about 23% of the sample.
- Live tracking and ETA is next, at about 13%. Alerts add about 6%.
- Driver and vehicle problems are about 9%, so some of the pain comes from operations and not from the app.
- About 90% of the sample is one-star. Many reviews are only a few words long.
- The monthly counts go up and down, but without trip numbers we cannot call it a trend.

Item	Detail
App	MoveInSync employee app for Android (com.moveinsync.ets)
Listing check	4.3 stars, about 1.15 lakh reviews, India view. Last app update 8 October 2026. Checked on 10 October 2026
Main data	500 one-star and two-star reviews, picked at random with my own script
Other sources	MoveInSync Help Center, and the case brief and job description I was given
Data used	Public data only. I did not use any internal MoveInSync data or private employee data

1. Why I did this

People use this app early in the morning, when they are trying to reach work on time. If a booking is wrong or the vehicle is missing from the map, they lose time, and sometimes money. The case brief says the rating needs to improve, so I started from there.

One average number can't tell us which problems matter most or whether the app is even the cause. I wanted to get as close to those answers as public data allows. My goals were:

- Collect reviews in a way someone else can repeat.
- Find the main complaint topics and see how big each one is.
- Turn what I found into things we can test, and put them in order.
- Write a 30-day plan that a team could start the next day.
- Set up a weekly routine, because someone will keep asking “how are our ratings?”

2. How I collected the data

I wrote a notebook that runs in Google Colab. It uses a Python package called google-play-scraper to ask for the public reviews of com.moveinsync.ets on the Google Play India store, in English. It looks at the newest reviews it can reach (up to 15,000), keeps only the written ones rated 1 or 2 stars, removes duplicates, and then picks up to 500 at random. I fixed the random seed at 42, so anyone who runs it gets the same kind of sample.

Step	What the script does	Why
1. Collect	Asks for the newest reviews (up to 15,000) from the India store, in English	So the starting pool is clear and can be repeated
2. Filter	Keeps written reviews rated 1 or 2 stars	We want to see what is going wrong
3. Remove duplicates	Uses the Play review ID, or a backup hash if the ID is missing	So no review is counted twice
4. Sample	Picks up to 500 at random, without repeats, using seed 42	To avoid cherry-picking and to make the run repeatable
5. Tag	Adds a topic tag using keyword rules	To turn free text into numbers we can count
6. Save	Exports a CSV with date, stars, app version, other details, review length, developer reply info, source details and tags. Also saves notes about the run	So others can check the work
7. Protect privacy	Leaves out reviewer names and the full review text	To keep personal data out

Table 1. The data collection steps.

Open the notebook:
https://colab.research.google.com/drive/1fP52lR-nGd6N0qryWATvWfgnzOhGsO0j?usp=sharing

To run it again, open the link and choose Runtime, then Run all. Please check the number of rows in the CSV you get. If Google Play shows fewer than 500 unique reviews, the real count is lower than 500, and the report should use the real count.

How I tagged the reviews

Each review gets one main topic: booking and scheduling; GPS, ETA and live tracking; alerts and notifications; driver and vehicle operations; crashes and speed; login and access; look and ease of use; customer support; or “Other / unclear”. The tags come from keyword rules, so they are quick and consistent, but they make mistakes. A review can mention more than one problem, so I treat a tag as a hint to look closer, not as a final answer. In the plan I include a step where a person checks about 50 tags by hand to see how often the rules are wrong.

What could go wrong with this data

- People who leave one or two stars are not a fair picture of all employees. This sample shows unhappy reviewers only.
- I used a community tool, not an official Google API. It can stop working or change what it returns.
- Keyword tags can be wrong, and about a third of the reviews have no clear topic.
- I did not divide monthly counts by trips or users, so they are counts and not complaint rates.
- A tag tells us what people say. It does not prove what caused the problem. For that we need internal data.

3. What the 500 reviews show

3.1 What people complain about

Figure 1 shows the main topic of each review. I read the bar lengths off the chart to make Table 2, so the numbers are close but not exact. They add up to about 500, which matches the sample size. I would replace them with the exact counts from the CSV before anyone quotes them.

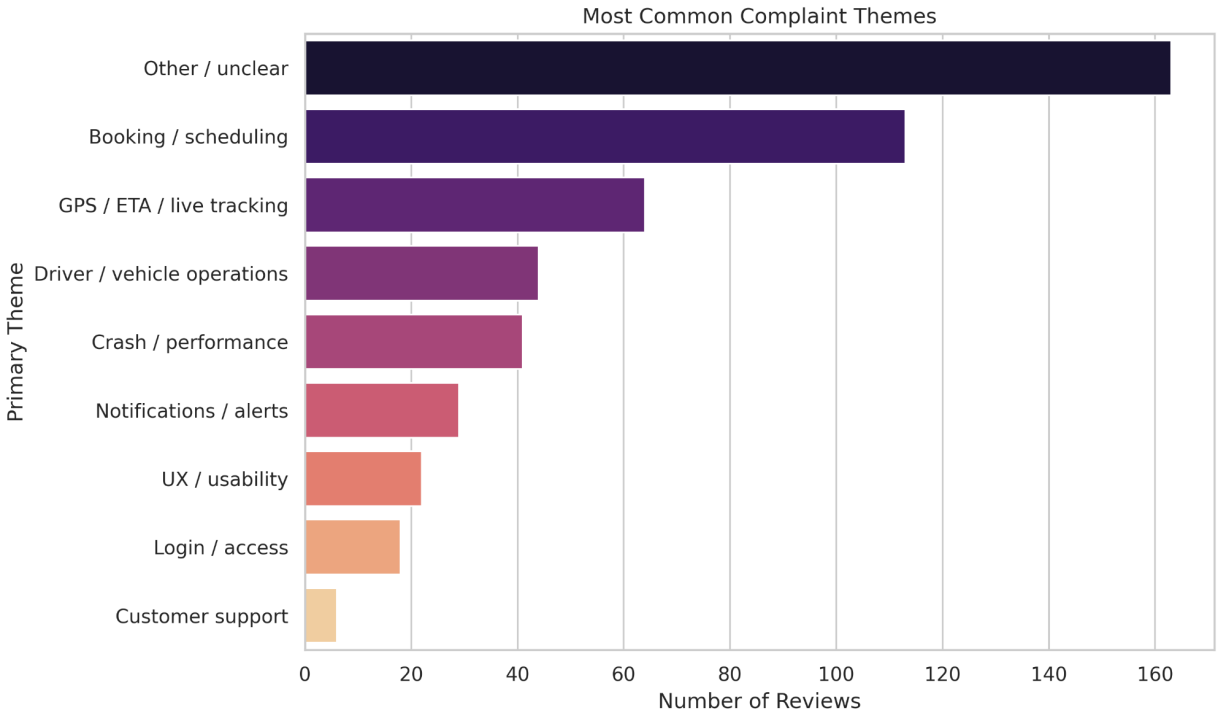


Figure 1. Main complaint topics in the 500-review sample.

Main topic	Reviews (approx.)	Share of 500 (approx.)
Other / unclear	163	33%
Booking / scheduling	113	23%
GPS / ETA / live tracking	64	13%
Driver / vehicle operations	44	9%
Crash / performance	41	8%
Notifications / alerts	29	6%

Main topic	Reviews (approx.)	Share of 500 (approx.)
Look and ease of use	22	4%
Login / access	18	4%
Customer support	6	1%

Table 2. Topic counts, read from Figure 1.

It is easier to see the pattern when the topics are grouped.

Group	Topics in it	Reviews (approx.)	Share (approx.)
Commute trust	Booking, GPS/ETA, alerts	206	41%
Operations	Driver and vehicle problems	44	9%
App working and sign-in	Crashes, speed, login	59	12%
Look and feel	Ease of use	22	4%
Support	Customer support	6	1%
No clear topic	Other / unclear	163	33%

Table 3. Topics grouped together. All numbers are approximate.

A few things stand out to me:

- Booking, tracking, alerts and operations together make up about half the sample. The look of the app is only a small part. This is why I would work on correctness and visibility before any redesign.
- Booking is the biggest topic we can name, at about one review in four. It is the first place I would look at real cases.
- Tracking is about half as common as booking, but it hurts at the worst moment, when someone is standing outside waiting for a ride.
- Driver and vehicle problems are a topic of their own. So part of the fix belongs to Operations, not to the code.
- The no-topic group is a third of the sample. I don't think those reviews are impossible to understand. I think many are just very short (see 3.3). Reading a sample of them is the first job in my plan.

3.2 When the reviews were posted

Figure 2 shows the number of low-rating reviews per month (top), and the total for each calendar month across all years (bottom). The sample goes from late 2019 to autumn 2026.

The counts move around a lot. The highest single month is about 20 reviews, and there are some very quiet stretches. Across all years, September is the highest month at a little under 60, followed by January, July and March. I would not read much into this yet. These are plain counts, and they depend on how many people use the app, how many trips run, when releases go out, and how well my script reached each period. The latest months may also be incomplete.

To make this chart useful, I would mark release dates and big operational changes on it, such as new shift patterns, and then compare low-star reviews with total reviews or trips in the same period. If spikes line up with a release, that is a lead. If they line up with busy travel months, it may just be demand.

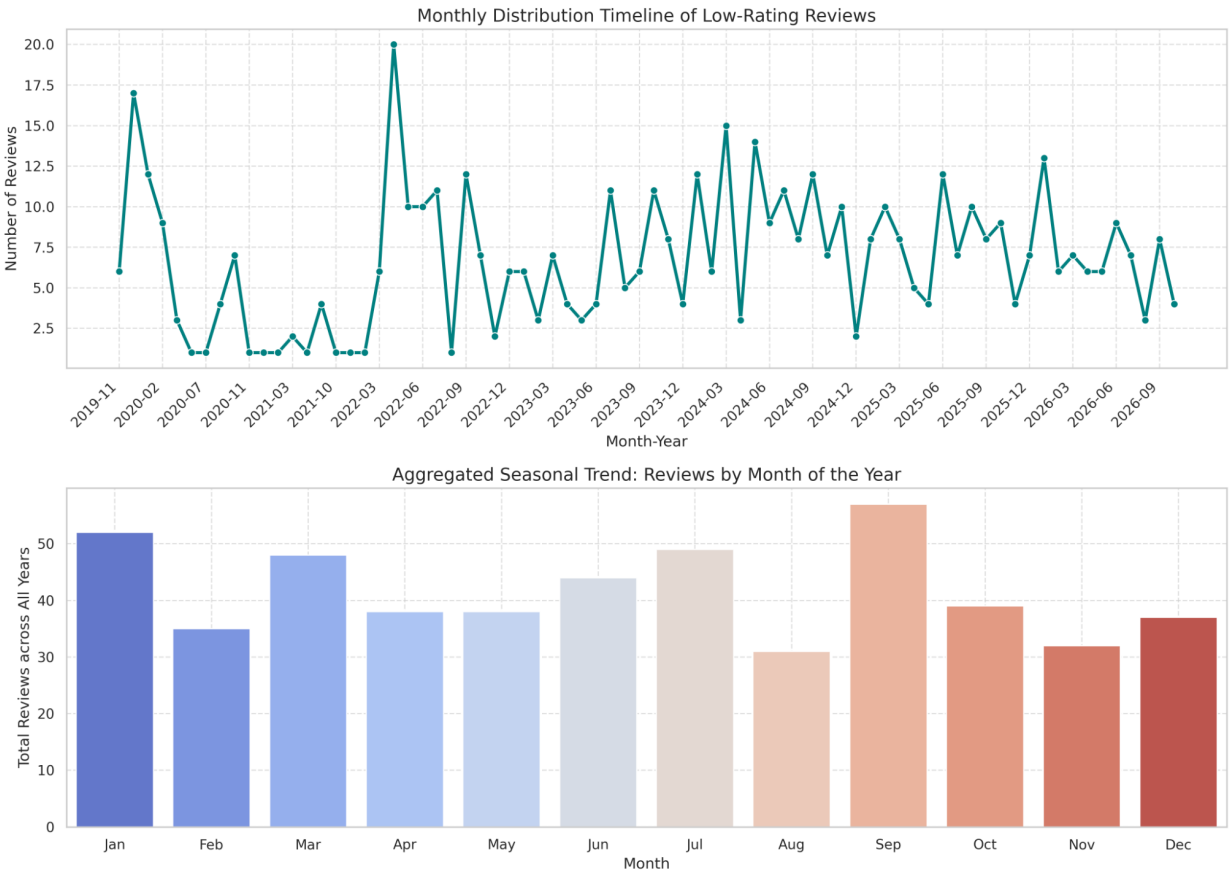


Figure 2. Low-rating reviews by month (top) and by calendar month across all years (bottom).

3.3 Star ratings and review length

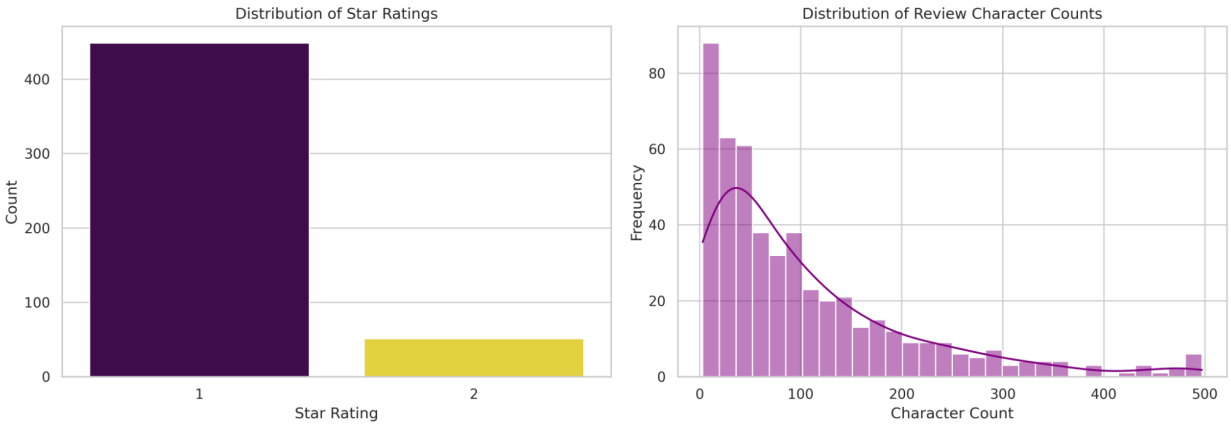


Figure 3. Star ratings (left) and review length in characters (right).

About 450 of the 500 reviews are one-star, and about 50 are two-star. Most reviews are short. The tallest bar is for the very shortest reviews, close to 90 of them, and the counts drop as reviews get longer.

This helps explain the no-topic group. A review with just a few words, like a single angry line, can't be tagged by any rule. I think short reviews make up a big part of the “Other / unclear” group. I can check that by comparing review length with topic, which is a quick test once I have the CSV. Longer reviews usually give more detail, but a short review about a missed pickup matters just as much as a long one, so I would check both kinds by hand.

3.4 What this data cannot tell us

It can tell us what unhappy reviewers say and when they say it. It cannot tell us what share of trips go wrong, which app versions are affected, or whether one complaint caused the low rating. For those we need internal trip, support and app data. That is why I list a day-one data request in Section 7.

4. What I think is going wrong

The case brief describes the full journey: onboarding, roster set-up, booking, vehicle assignment and routing, pickup time messages, live tracking, OTP and driver safety checks, finishing the trip, and raising issues. The app is where most of that shows up for the employee. Table 4 matches the topics from the sample to those steps.

Journey step	What the employee needs	Matching topic in the sample
1. Book	The right date, time and cab or shuttle, with a clear confirmation	Booking / scheduling (23%); ease of use (4%)
2. Get assigned and informed	A vehicle, a pickup time and alerts on time	Alerts (6%); driver / vehicle operations (9%)
3. Wait and board	A fresh vehicle location, an ETA they can trust, and a clear check-in and no-show status	GPS / ETA / live tracking (13%)
4. Finish the trip	A correct trip record, and a way to raise problems	Customer support (1%). No-show is not tagged on its own
All the time	An app that opens and signs in	Crash / performance (8%); login (4%)

Table 4. Review topics matched to the commute journey.

My main idea is that the biggest topics are all cases where the app and the real trip do not agree: what was booked, where the vehicle is, when it will arrive, and whether an alert came in time. I want to test that with event-level data.

There is also a gap. My tags have no separate topic for no-show or check-in disputes, so those complaints may be hiding inside other groups. A wrong no-show can cost someone money, so this is the first tag I would add.

Other explanations I need to rule out

- A late vehicle, weak GPS signal or a poor network may be an operations problem and not an app bug.
- A booking screen may have been changed on purpose but not explained well. Usability tests can tell the difference between confusing design and a booking that really failed to save.
- An alert can leave the server and still never be seen, because of phone permissions, battery settings or network. Sent is not the same as seen.
- Many complaints may come from one site or one shift group and not from everyone.

Idea to test	What the 500 reviews show	Data to look at	How to test it	Order
H1. What the employee booked and what was saved can be different	Booking is the biggest named topic (about 23%)	Chosen date, time and type; booking ID; confirmation; retries; the saved record	Follow sample cases from the tap to the saved trip. Compare what the user chose, what was saved, what the screen said and which trip was assigned	P0

Idea to test	What the 500 reviews show	Data to look at	How to test it	Order
H2. No-show or check-in status can be a surprise or wrongly applied	No separate tag, so we cannot measure it yet	Status changes, check-in events, waiting time rules, GPS near pickup, driver timestamps, disputes	Add a tag. Review every status change. Count the no-shows that were wrong	P0
H3. Vehicle location and ETA can be old or wrong	GPS / ETA about 13%	Time the GPS point was made versus received, location age, ETA error, device, network	Measure how old locations are and how far ETA is off, by trip, route and time	P1
H4. Important alerts can be late or not useful	Alerts about 6%	Time of trigger, response from push service, device token, retries, permissions	Measure delay at each step. Find where alerts fail. Test an in-app trip timeline as a backup	P1
H5. A changed booking screen makes people unsure	Ease of use about 4%, and it overlaps with booking	Steps completed, taps, confirmation screen, drop-offs	Short usability tests. Compare before and after on a small group	P2
H6. Crashes or sign-in problems stop people using the app	Crashes about 8%, login about 4%	OTP success, login time, crashes, launch success, phone model, OS, app version	Break down failures and check if they match low-star reviews	Parallel

Table 5. Ideas to test, in order. The order is based on how much harm is possible and how urgent the moment is. It is not based on how sure I am about the cause.

Help Center check. MoveInSync's Help Center has pages on login failures, OTP delivery, crashes, launch and loading problems, opt-in and opt-out visibility, and network issues. That shows these problems are known. It does not show how often they happen or how much they hurt the rating.

5. My 30-day execution plan

This is the most important part of the report. I am not promising a big rating jump in 30 days. What I want is to find the failures that matter most, fix one or two of them once they are confirmed, and leave behind a routine the team can keep running each week.

I would follow these rules:

- Prove a problem before building a fix. A review alone is not enough.
- Anything that touches money, safety or a missed shift comes first, even if it happens less often.
- Make small changes that can be undone, and give them to a small group first.
- Watch the early signs, like correct bookings and fresh locations. The star rating moves last.
- Talk to the people who run the service every day. Operations, Support and drivers know things no dashboard shows.

5.1 Who does what

There are five workstreams. Each has one person who is accountable, so nothing falls between teams. In Table 6, A means accountable, R means does the work, C means is asked for input, and I means is kept informed.

Workstream	What it covers	Product	Mobile	Backend	QA	Ops / Support	FDE
WS1 Review analysis	Re-run the notebook, fix tags, check tag accuracy, weekly report	C	I	I	I	C	A / R
WS2 Booking	H1 and H5: audit, tests, fix	A	R	R	R	C	R
WS3 No-show	H2: rules, audit, appeals	A	C	R	R	R	R
WS4 Tracking and alerts	H3 and H4: location age, ETA, alert path	C	R	R	R	C	A
WS5 Measuring and release	Numbers, dashboards, safe rollout	C	R	R	C	I	A / R

Table 6. Who is responsible for what. Names get filled in on Day 1.

5.2 The plan, step by step

Each phase has a clear “gate”. We only move on when the gate is met. If it is not, we stop and think again.

Phase and days	What I would do	What we have at the end
Phase 0 **Days 1–2** **Get aligned**	Hold a short kick-off with Product, Support, Operations, Mobile and Backend. Agree what “booking mismatch”, “stale location” and “wrong no-show” mean. Share the review topics and ask which ones match cases the team already knows about. Confirm the app and package. Add tags for no-show and check-in. Re-run my notebook for a fresh sample, then check about 50 tagged reviews by hand, including some from “Other / unclear”, to see how accurate the tags are.	A shared problem list, agreed definitions, named owners, better tags and a list of data we are missing. **Gate:** Product and Operations sign off the definitions.
Phase 1 **Days 3–5** **Trace real cases**	Take a small set of real cases, with personal details removed, and follow each one from the employee's action to the saved booking, the driver and vehicle update, and the alert. Start with booking confirmation and no-show disputes, because the cost to the employee can be high. Write the first queries for booking mismatch, location age, ETA error and alert delay, where data exists.	A few cases traced from start to end, first baseline numbers, and a clearer idea of which hypotheses hold. **Gate:** at least one hypothesis is confirmed or rejected with evidence.
Phase 2 **Days 6–7** **Decide**	Go through the evidence with the people closest to the work. Separate app bugs from network, phone, driver and operations causes. Pick the smallest fix with the biggest effect. Agree how we test it, what we watch, and when we roll back. No big redesign.	A short ranked list of work, one or two fixes chosen, tests and rollback rules agreed. **Gate:** Product and Engineering approve the fix.
Phase 3 **Week 2** **Build**	Build the fix behind a feature flag or a staged release. Write tests for the exact failure we saw. Add tracking events so we can tell if the change worked. Set up the dashboard and the comparison of support contacts.	A tested change, a release and rollback plan, and a dashboard showing the result we care about. **Gate:** tests pass and the dashboard shows live data.

Phase and days	What I would do	What we have at the end
Phase 4 Week 3 Release	Release to a small group first. Check every day: booking correctness, no-show disputes, location age, alert delay, crashes and support contacts. Talk to Operations right away if numbers move in a way we did not expect. Write down the decision.	A written decision to expand, adjust or roll back, with the evidence. **Gate:** guardrails held for the agreed watch period.
Phase 5 Week 4 Learn	Compare the numbers after release with the baseline. List other changes that could explain the movement. Share what worked and what did not. Turn the remaining topics into next sprint's work. Hand the weekly review to a permanent owner.	A short readout, a list of next steps and a weekly review that keeps running. **Gate:** leadership readout done.

Table 7. Phases and gates.

5.3 The 30 days at a glance

Workstream	Days 1–2	Days 3–5	Days 6–7	Week 2	Week 3	Week 4
WS1 Review analysis	Fix tags	Check tags	First report	Weekly	Weekly	Hand over
WS2 Booking	Define	Trace cases	Pick fix	Build, test	Small release	Evaluate
WS3 No-show	Define	Trace cases	Pick fix	Check rules	Small release	Evaluate
WS4 Tracking, alerts	Define	Baseline	Rank	Add events	Watch	Evaluate
WS5 Measuring	Definitions	Queries	Dashboard	Guardrails	Daily watch	Readout

Table 8. Timeline. Booking and no-show go first because the cost to employees is highest.

5.4 What the app and backend need to record

I can't test most of my ideas without a shared record of what happened at each step. I suggest a small set of events. Where the system already logs something, we reuse it, and we only add what is missing. Each event carries a trip or booking ID that hides the person's identity, a server time, a phone time when it matters, the app version and the platform. We do not need names or free text.

Step	Event	Why we need it	Idea
Book	booking_submitted	Records what the employee chose (date, time, type) at the moment they tapped	H1, H5
Book	booking_persisted	Shows what the backend actually saved, to compare with the line above	H1
Book	booking_confirmation_shown	Shows what the employee saw, so we can find confusing screens	H1, H5
Inform	trip_assigned	Links the booking to a vehicle, route and pickup time	H1, H3
Inform	notification_triggered, provider_accepted, displayed	Tells us if the delay is on our server, the push service or the phone	H4
Wait	location_received, location_displayed	Gives the age of the location when the employee looked at it	H3
Wait	eta_computed, arrival_observed	Lets us work out how wrong the ETA was	H3
Board	checkin_recorded	Shows if and when the employee checked in	H2
Finish	noshow_decided (with the rule and inputs)	Lets us explain every no-show decision	H2

Step	Event	Why we need it	Idea
Finish	noshow_disputed, noshow_resolved	Tells us how often no-shows were wrong and how long fixing them takes	H2
Any time	app_crash, otp_result, login_result	Baseline for crashes and sign-in	H6

Table 9. Minimum set of events.

5.5 Releasing safely

A mistake in booking or no-show logic can hurt employees directly, so I would never release a fix to everyone at once. I have left the threshold values empty on purpose. They should come from the baseline numbers in Phase 2. I don't want to guess them now.

Stage	Who gets it	What we watch	Move on if	Roll back if
0. Inside the team	Testers and QA, using made-up trips	Test suite and manual checks	All tests pass	Any confirmed serious mismatch
1. Small group	One site or shift group, agreed with Operations	Daily numbers and support contacts	No guardrail is broken for the agreed period	A guardrail is broken, or Operations raises a safety concern
2. Bigger group	Several sites, or a larger share of users	The same numbers, compared with a group that did not get the change	It keeps improving and nothing gets worse	Worse than the comparison group by more than the agreed limit
3. Everyone	All users	Weekly review, then normal monitoring	Stable for a full week	Any serious mismatch. Rollback is already approved

Table 10. Staged release.

- Numbers that must not get worse: booking mismatch, crash-free sessions, support contacts per thousand trips, no-show disputes and alert delay.
- I would not touch OTP, driver checks or incident escalation without a clear sign-off from Operations and Security.
- One engineering owner decides whether to expand or roll back, and writes the reason in the decision log.

5.6 How I would measure success

The star rating moves last, so I would track the early signs first. For each number, the team should write down who owns it, the query it comes from, the time period, what is left out, and what an alert should trigger. We set targets only after we know the baseline.

Measure	Plain meaning	Comes from	How often
Booking mismatch rate	Bookings where the saved date, time or type is not what the user meant, divided by bookings checked (exclusions written down)	booking_submitted compared with booking_persisted	Daily
Old-location rate	Tracking events where the location is older than an agreed limit, divided by all relevant tracking events	location_received times	Daily
ETA error	How far the predicted arrival time was from the real one, at an agreed checkpoint	eta_computed and arrival_observed	Daily or weekly

Measure	Plain meaning	Comes from	How often
Alert delay	Time from the trigger to the delivery step we can measure. Accepted by the push service does not mean the user saw it	notification events	Daily
Wrong no-show rate	No-shows found to be wrong after review, divided by no-shows reviewed	noshow_decided plus review	Weekly
Low-star share	New 1-star and 2-star reviews divided by all new reviews in the same period. Read it together with trip numbers	Review notebook	Weekly

Table 11. The measures.

How I would judge the results

- Get a baseline first, ideally several weeks, or as much history as exists.
- Compare like with like: the same days of the week and the same time window, and a comparison group where possible.
- Write down anything else that could move the numbers: other releases, festivals, weather, driver roster changes.
- If a group is small, say so. A result that points in a direction is still useful, but it is not proof.
- Do not chase the star count. It is fine to ask for feedback after a finished trip, within Google's rules. Negative feedback must never be hidden, and nobody should be interrupted while booking, waiting for pickup or dealing with an incident.

5.7 Keeping everyone in step

I would run a 15-minute check-in every week with four questions. What did we learn? What changed for employees? What are we still not sure about? Who owns the next action, and by when? If the evidence does not support a fix, we say so and change the hypothesis. We do not force a fix to fit.

Meeting	When	Who	Purpose
Weekly check-in	Every week, 15 minutes	Product, Mobile, Backend, Ops, FDE	Progress, blockers, next steps
Evidence review	Day 5 and Day 7	Workstream owners, Operations lead	Confirm or drop hypotheses, choose fixes
Release review	Each release	Engineering owner, QA, Operations	Expand, adjust or roll back
Leadership readout	Day 30	Product and Operations leaders	What improved, what did not, what is next

Table 12. Meetings.

Every decision goes into a short log with the date, the evidence, the owner and the result. That way the reasoning is still there when people change roles, and anyone can check it later.

5.8 Risks and how I would handle them

Risk	Chance / impact	What I would do	Owner
Internal data comes late or is incomplete	Medium / High	Start with public data and made-up traces. Ask for limited-access exports on Day 1. Write the gaps down	FDE
Keyword tags label reviews wrongly	High / Medium	Check a sample by hand every week, track the error rate, improve the tags	FDE
A fix affects safety or pay-related flows	Low / Very high	Small groups first, rollback agreed beforehand, sign-off from Operations and Security	Engineering owner
Other releases confuse the before-and-after comparison	Medium / Medium	Mark releases on the timeline, use a comparison group, note other changes	FDE
The scraping tool changes or stops working	Medium / Low	Keep the notebook in version control. Use an approved export if one is available	FDE
Privacy concerns with review or trip data	Low / High	Hide identities, keep only what is needed, never send sensitive text to outside AI tools	FDE, Security
Teams feel the plan is blaming them	Medium / Medium	Present findings as ideas to test, share the evidence openly, credit the teams who fix things	FDE, Product

Table 13. Risks.

5.9 What “done” looks like after 30 days

- Shared definitions, and better review tags that a person has checked.
- At least one booking or no-show problem followed from start to end, and either confirmed or ruled out.
- A baseline for each early-sign measure, with the missing data listed honestly.
- One or two fixes released step by step, each with a written decision to expand, adjust or roll back.
- A weekly review routine with a named owner and a decision log.
- A short readout for leaders that separates what we proved, what we suspect and what we still don't know.

6. A weekly routine for “how are our ratings?”

Someone will keep asking this question, so I would turn the answer into a short weekly routine and not a one-time document. It has six steps: collect the new reviews and the listing numbers; clean them (dates, ratings, app version, duplicates); tag them by topic with a person checking some; score each topic by how bad it is, how often it happens and how sure we are, and raise it if it touches safety, money or something that cannot be undone; match topics to trip and support data when we are allowed to; and write a short weekly summary with owners and release notes.

When	What happens	Who
Monday	Refresh the rating and reviews. Tag new reviews. Compare the last four weeks	Analyst or FDE. Keep the date and app on every record
Tuesday	Look at top topics next to incidents. Give each an owner, priority, due date and next test	Product, Engineering, Support, Operations

When	What happens	Who
Each release	Note the date and version. Compare early-sign numbers and review topics. Watch the guardrails	Engineering owner decides to expand or roll back
Monthly	Sum up what got better, what did not, open risks and ideas for next quarter	Shared with Product and Operations leaders

Table 14. The weekly routine.

Handling the data carefully

- Use public reviews in line with Google Play's terms. If there is an approved API or export, use it instead of scraping.
- Don't send personal or sensitive review text to outside AI services. Remove what is not needed.
- Keep source links, dates, labels and any manual changes, so anyone can check how a review was tagged.
- Treat AI or rule-based tags as suggestions. Track the mistakes and check a sample by hand.
- Make the code handle pages that don't load, duplicates, missing star ratings and empty data.

7. Limits, data request and conclusion

Limits

- The 4.3 rating and 1.15 lakh review count are a snapshot from one day. They are not a trend or a target.
- The 500 reviews are only written one-star and two-star reviews that the source returned. They show unhappy reviewers, not all employees.
- The counts and the star split in this report are read from the charts, so they are close but not exact. I would confirm them from the CSV.
- The tags come from keyword rules, and about a third of reviews have no clear topic.
- Monthly counts have no trip numbers behind them, and the newest months may be incomplete.
- Public reviews can be wrong or incomplete. Causes and rates stay unproven until we match them with event logs, support data and operations data.
- The Help Center article lists topics only. It does not show how often they happen.
- I did not invent targets. Thresholds and goals need internal baselines.

What I would ask for on day one

A 90-day review export with dates and ratings, with identities removed; app version and phone details; booking and trip events; GPS and ETA timestamps; push and delay logs; crash-free session numbers; support ticket tags; and no-show disputes with their outcomes. Access should be limited to what is needed, with hidden identifiers and no more personal data than necessary.

Conclusion

The reviews give a clear direction, but not a final answer. Booking, tracking and no-shows are where employees say the app and the real trip do not match, and the 500-review sample backs this up for the first two. My plan is to confirm the failures with real event data, fix the proven ones in small steps, and then measure two things: whether the commute got better, and whether the reviews followed.

References

[1] Google Play, MoveInSync employee app listing (India), package com.moveinsync.ets. Checked 10 October 2026: 4.3 stars, about 1.15 lakh reviews, updated 8 October 2026.

[2] MoveInSync Help Center, “Mobile App Issues”, last modified 14 May 2025.

[3] Candidate assignment and Forward Deployed Engineer job description given during this hiring process.

[4] Review collection notebook (Google Colab), using the google-play-scraper package, India store, English, 1 and 2 star written reviews, random sample of up to 500 with seed 42: https://colab.research.google.com/drive/1fP52lR-nGd6N0qryWATvWfgnzOhGsO0j?usp=sharing